# Parcial práctico de Ingeniería de Software: La última transmisión

**Motor de historias interactivas.** Informe de la entrega (cuerpo de 8 páginas como máximo; los diagramas, la matriz de trazabilidad y las capturas del tablero están en `docs/Anexo.pdf`).

| Integrante | Código |
|---|---|
| Camilo Conde Corrales | T00078909 |
| Miguel Jacome y Yanez | T00078937 |

**Versiones:** `v1.0` (previa a la solicitud de cambio) y `v2.0` (final). **Repositorio:** código, datos, pruebas y evidencias incluidos en el ZIP. Instrucciones de ejecución en `README.md`.

## 1. Fundamentos: problema, actores y alcance

**Problema.** Un colectivo de escritores prepara una ficción interactiva para un festival universitario. Necesita un programa que interprete relatos escritos en archivos, ofrezca decisiones que dependen de lo ocurrido en la partida y avise de los errores del contenido antes de que un lector lo juegue. Hoy cualquier cambio en la historia exigiría modificar código, y un archivo mal escrito podría cerrar el programa frente al público.

**Actores.**

| Actor | Rol frente al sistema | Interés principal |
|---|---|---|
| Escritora (autora de contenido) | Escribe y edita archivos de historia JSON; ejecuta la validación | Cambiar escenas y decisiones sin programar (N01, N06) |
| Diseñador de la experiencia | Define condiciones y efectos de las opciones | Decisiones con consecuencias (N02, N03, N04) |
| Lector (jugador) | Recorre la historia en consola | Leer, elegir, reconocer el final y reiniciar (N02, N05) |
| Coordinadora del festival | Opera el programa en el evento | Que un error no cierre el programa de forma inesperada (N06) |

**Alcance de la versión entregada.** Incluye: carga de historias desde archivos JSON con un contrato documentado; validación previa con mensajes que indican la causa y el campo o ID; motor de sesión con condiciones de igualdad booleana y efectos de asignación; interfaz de consola, reinicio, detección de bloqueo durante la partida y deshacer la última decisión (solicitud de cambio SC01). **Fuera de alcance:** cuentas, base de datos, guardado entre ejecuciones, red, editor visual, audio, imágenes, lenguaje de expresiones y análisis automático de todos los caminos del relato.

**Riesgos y mitigación.**

| ID | Riesgo | Prob./Impacto | Mitigación aplicada |
|---|---|---|---|
| R1 | Un archivo con errores (destino inexistente, variable mal escrita) provoca un cierre inesperado durante el festival | Media / Alto | Validación completa **antes** de abrir la sesión; la interfaz atrapa `ErrorContenido` y muestra la lista de problemas; pruebas U4 parametrizadas con las nueve categorías |
| R2 | La lógica de la historia de referencia queda "cableada" en el código y la segunda historia no funciona sin editar el motor | Media / Alto | El motor evalúa condiciones y efectos solo desde los datos; prueba de modificabilidad con una segunda historia de IDs y variables distintos; revisión de que `motor/` no contiene IDs como `E1` o `conoce_origen` |

**Priorización de atributos de calidad.** Se ordenaron según las voces del cliente y la rúbrica:

1. **Confiabilidad** (coordinadora: "el lector no debe encontrarse con un cierre inesperado"). Una entrada inválida no puede alterar ni terminar la partida.
2. **Modificabilidad del contenido** (escritora: "agregar escenas sin pedir que modifiquen el programa"). Es la razón de existir del motor.
3. **Claridad de errores** (N06). Sin ella la escritora no puede corregir sola su archivo.
4. **Facilidad de uso** de la consola: importante, pero con menor riesgo porque la interacción es mínima (elegir un número).

El rendimiento no se priorizó: las historias tienen decenas de escenas y la evaluación es lineal.

**Decisión de responsabilidad profesional (autoría y dependencias).** El contenido narrativo pertenece al colectivo de escritores; el motor no modifica, recolecta ni transmite los textos ni las decisiones de los lectores (no hay telemetría ni archivos de registro de partidas). Además, se decidió **no usar dependencias en tiempo de ejecución**: el motor solo usa la biblioteca estándar de Python (licencia PSF)

y la única dependencia de desarrollo es `pytest` (licencia MIT), documentada en el README. Así el festival puede ejecutar el programa sin descargar paquetes de terceros y sin obligaciones de licencia adicionales.

## 2. Proceso y agilidad

**Proceso elegido: Kanban incremental con dos entregas internas.** El trabajo dura 12 horas en pareja y llega un cambio en la hora 6. Se usa un tablero con flujo continuo, límite de trabajo en curso (WIP = 2, una tarjeta por integrante en "En progreso") y dos versiones etiquetadas en Git: `v1.0` (versión disponible al recibir el cambio) y `v2.0` (versión final).

**Comparación con la alternativa (Scrum).** Scrum organiza el trabajo en sprints de duración fija con planificación, revisión y retrospectiva. En una ventana de 12 horas, un sprint útil ocuparía casi toda la ventana y sus ceremonias consumirían tiempo de construcción; además, el cambio de la hora 6 llega a mitad de cualquier sprint razonable. Kanban permite reordenar el backlog en el momento en que llega la solicitud sin "romper" un compromiso de sprint.

| Actividad | Cómo se realiza en esta entrega |
|---|---|
| Especificación | RF y RNF propios en este informe, con criterios de aceptación derivados de N01-N06 y de CA01-CA03 |
| Desarrollo | Incremental por tarjetas del tablero: carga y validación, luego motor, luego interfaz, luego cambio |
| Validación | Pruebas automatizadas (pytest) escritas junto a cada incremento; recorridos manuales de los finales; script de verificación de RNF |
| Evolución | Tag `v1.0` antes del cambio; análisis de impacto (sección 7); actualización de backlog, modelos, código, pruebas y matriz; tag `v2.0` |

**Backlog priorizado** (el tablero y sus capturas inicial y final están en el Anexo, sección A9).

| ID | Elemento | Prioridad | Responsable |
|---|---|---|---|
| B01 | Alcance, actores, riesgos y calidad | Alta | Camilo Conde |
| B02 | Requisitos RF/RNF y preguntas al cliente | Alta | Camilo Conde |
| B03 | Revisión cruzada de requisitos | Alta | Miguel Jacome |
| B04 | Modelos (contexto, casos de uso, clases, secuencia, estados) | Alta | Camilo Conde |
| B05 | Arquitectura y decisiones (ADR) | Alta | Camilo Conde |
| B06 | Cargador y validador con mensajes de error | Alta | Camilo Conde |
| B07 | Motor de sesión: opciones, elección, estado, reinicio | Alta | Camilo Conde |
| B08 | Interfaz de consola | Media | Camilo Conde |
| B09 | Pruebas U1-U5 e I1 | Alta | Camilo Conde |
| B10 | Segunda historia y su redacción | Media | Miguel Jacome |
| B11 | Verificación de RNF y ejecución en un segundo equipo | Media | Miguel Jacome |
| B12 | (Cambio SC01) Deshacer la última decisión, U6 y extensión de U5 e I1 | Alta | Camilo Conde |
| B13 | (Cambio SC01) Actualizar modelos, matriz y README | Alta | Camilo Conde |
| B14 | Video de demostración y empaquetado | Alta | Ambos |

**Prácticas ágiles aplicadas.**

1. **Programación en pareja con rol de revisor.** Mientras un integrante implementaba un incremento, el otro revisaba requisitos y pruebas; la revisión cruzada de requisitos (sección 3.4) detectó un defecto antes de codificar.
2. **Pruebas primero en el motor (TDD ligero).** Los escenarios U1-U5 se escribieron a partir de los criterios de aceptación antes o junto con el código de `motor/sesion.py`. Esto obligó a que `elegir` devolviera un resultado verificable en lugar de imprimir en pantalla.

**Alternativa descartada: estimación con *planning poker*.** Con 14 elementos y dos personas, estimar con cartas tomaba más tiempo que la tarea; se usó la distribución de tiempo sugerida por la guía como límite por bloque.

**Definición de terminado.** Una tarjeta pasa a "Hecho" cuando: (1) el código está en `main`; (2) tiene pruebas automatizadas que pasan con `python -m pytest`; (3) los mensajes al usuario están en español y no muestran trazas de Python; (4) los requisitos, modelos y matriz que la mencionan están actualizados; (5) el otro integrante la revisó.

**Aprendizaje y acción de mejora.** Al recibir el cambio de la hora 6 comprobamos que haber separado en `Sesion` la validación de la opción (`_opcion_habilitada`) de la modificación del estado (`_aplicar`) permitió agregar deshacer en un solo punto: guardar la instantánea justo antes de `_aplicar`. Esa separación salió del diseño pensado para las pruebas U3, no de prever el cambio. Además, la verificación automatizada de RNF03 encontró un defecto (`'²'`) que las pruebas escritas a mano no habían cubierto. **Acción de mejora:**

en el próximo proyecto, desde el primer incremento (y no al final) agregaremos una prueba de entradas aleatorias con semilla fija para cada punto donde el sistema recibe datos del usuario.

# 3. Requerimientos

## 3.1 Requisitos funcionales

Cada RF describe un comportamiento observable y su criterio de aceptación (CA). "Historia de referencia" es `historias/la_ultima_transmision.json`.

| RF | Necesidad | Requisito | Criterio de aceptación |
|---|---|---|---|
| RF01 | N01 | El sistema carga una historia desde la ruta de un archivo JSON indicada al ejecutar el programa. | `python jugar.py historias/el_faro.json` muestra el título y la escena inicial de ese archivo, no los de otra historia. |
| RF02 | N01 | Al comenzar una partida, la escena actual es la indicada en `inicio` y cada variable toma el valor declarado en `variables`. | Con la historia de referencia, una sesión nueva está en `E1` con `conoce_origen = falso` y `confia_en_la_voz = falso`. |
| RF03 | N02 | El sistema muestra el texto de la escena actual. | Al iniciar la historia de referencia se muestra el texto de `E1`. |
| RF04 | N02 | El sistema muestra **únicamente** las opciones de la escena actual cuyas condiciones se cumplen todas; un mapa de condiciones vacío habilita la opción. | En `E3` con `conoce_origen = verdadero` se muestran exactamente "Activar con código" y "Cortar comunicación" (CA01); con `conoce_origen = falso`, exactamente "Activar sin verificar" y "Cortar comunicación". |
| RF05 | N03 | Al elegir una opción habilitada, el sistema aplica todos sus efectos y cambia la escena actual al destino de la opción. | Desde `E2`, elegir `investigar` deja la sesión en `E3` con `conoce_origen = verdadero` (CA01). |
| RF06 | N03 | Si se solicita una opción inexistente o no habilitada, el motor la rechaza, informa el motivo y no modifica escena ni variables. | En `E3` sin conocer el origen, enviar `activar_codigo` directamente al motor devuelve rechazo y la sesión sigue en `E3` con las mismas variables (CA02). |
| RF07 | N03 | La interfaz rechaza entradas no reconocidas (texto libre, número fuera de rango, vacío) con un mensaje y sin modificar la partida. | En `E1`, escribir `9` muestra "Entrada no válida..."; escribir `abc` muestra "La opción 'abc' no existe en esta escena"; en ambos casos la sesión sigue en `E1` con las mismas variables. |
| RF08 | N04 | Los valores de las variables se conservan durante toda la partida y las condiciones posteriores se evalúan con ellos. | Tras `revisar` → `investigar`, en `E3` aparece "Activar con código" y no "Activar sin verificar". |
| RF09 | N05 | Cuando la escena actual es final, el sistema muestra su texto, la indica como desenlace y no ofrece opciones de avance. | Al llegar a `E6` se muestra "FIN" con el texto de `E6` y solo los comandos reiniciar y salir. |
| RF10 | N05 | El lector puede reiniciar en cualquier momento; la sesión vuelve a la escena inicial y a los valores iniciales de todas las variables. | Tras llegar a `E4`, reiniciar deja la sesión en `E1` con todas las variables en falso (U5). |
| RF11 | N06 | El sistema valida el archivo completo antes de abrir una sesión; si existe algún error, no se abre una sesión jugable. | Una copia con destino `E99` se rechaza y no se muestra ninguna escena (CA03). |
| RF12 | N06 | Cada error de contenido se informa con su categoría, la causa y el campo o ID involucrado. | El error de CA03 indica `[DESTINO_INEXISTENTE]`, la escena, la opción y el valor `E99`. |
| RF13 | N06 | El escritor puede validar un archivo sin jugarlo mediante `--validar`; el programa termina con código 0 si es válido y 1 si no lo es. | `python jugar.py --validar historias/invalidas/destino_inexistente.json` lista el error y termina con código 1. |
| RF14 | N06 | Si durante una partida una escena no final no tiene opciones habilitadas, el sistema informa el bloqueo y permite reiniciar o salir. | Con `historias/invalidas/bloqueo_en_partida.json`, al llegar a la escena bloqueada se muestra "BLOQUEO" y reiniciar vuelve al inicio. |

### 3.2 Requisitos no funcionales medibles

| RNF | Atributo | Requisito y métrica | Procedimiento de verificación | Evidencia |
|---|---|---|---|---|
| RNF01 | Modificabilidad del contenido | Incorporar una historia nueva o agregar una escena y una opción a una existente requiere **0 líneas modificadas** en `motor/` e `interfaz/`. | (a) Cargar y recorrer hasta todos sus finales la segunda historia `el_faro.json`; (b) el script agrega por programa una escena `E7` y una opción a una copia temporal de la historia de referencia y la juega; (c) compara el hash SHA-256 de `motor/` e `interfaz/` antes y después; (d) `git log --stat` del commit que añadió la segunda historia solo toca `historias/`. | `tests/test_rnf.py::test_rnf01_*`, `evidencias/rnf_resultados.txt` |
| RNF02 | Claridad de errores | El **100 %** de los archivos de `historias/invalidas/` produce con `--validar`: código de salida 1, un mensaje con la categoría `[...]` y el ID o campo esperado, y **0** trazas `Traceback`. | El script ejecuta `jugar.py --validar` como proceso externo sobre cada archivo y comprueba los tres criterios. | `tests/test_rnf.py::test_rnf02_*`, `evidencias/rnf_resultados.txt` |
| RNF03 | Confiabilidad | Ante **1000** entradas aleatorias de la interfaz (semilla fija 2026; texto, números, IDs, comandos), hay **0** excepciones no controladas y cada entrada rechazada deja escena y variables idénticas. | El script alimenta la interfaz de consola con las entradas generadas y compara el estado antes y después de cada entrada rechazada. | `tests/test_rnf.py::test_rnf03_*`, `evidencias/rnf_resultados.txt` |

### 3.3 Preguntas al cliente, respuestas y supuestos

| ID | Pregunta | Respuesta o supuesto registrado |
|---|---|---|
| P1 | La escritora quiere retroceder siempre y el diseñador quiere decisiones irreversibles. ¿El retroceso debe estar disponible siempre, limitarse a cierto número de pasos o prohibirse en decisiones marcadas como irreversibles? | **Acuerdo provisional (v1.0):** la primera versión no implementa retroceso; no se presupone la decisión final del cliente. **Respuesta (SC01, hora 6):** un solo nivel de deshacer que revierte todas las consecuencias. Ver RF15-RF17 en la sección 7. |
| P2 | ¿Las opciones cuyas condiciones no se cumplen deben verse deshabilitadas o no deben verse? | **Supuesto:** no se muestran (N02: "ver únicamente decisiones compatibles"; CA01 exige ver exactamente dos opciones en `E3`). |
| P3 | ¿El lector debe ver los valores de las variables durante la partida? | **Supuesto:** no; revelaría la lógica del relato. Para pruebas y demostración existe la opción `--depurar`, que muestra el estado. |

### 3.4 Revisión cruzada: defecto detectado y corrección

Los requisitos se revisaron entre integrantes contra N01-N06 y CA01-CA03 antes de implementar el motor.

| Defecto | Hallazgo | Corrección |
|---|---|---|
| D1 (ambigüedad / contradicción) | La primera redacción de RF04 decía "muestra las opciones de la escena actual **e indica cuáles están disponibles**". Admitía mostrar también las opciones no habilitadas (por ejemplo, en gris), lo que contradice N02 ("ver **únicamente** decisiones compatibles") y CA01, que exige ver **exactamente dos** opciones en `E3`. Además su criterio de aceptación no permitía comprobar que la opción no habilitada quedara oculta. | RF04 se reescribió como "muestra **únicamente** las opciones cuyas condiciones se cumplen todas" y su criterio ahora verifica los dos casos de `E3` con conjuntos exactos. Se alineó con el supuesto P2. Cambio visible en el historial de Git (commit "Corrige RF04 tras revisión cruzada de requisitos"). |

## 4. Modelado

Los modelos están en el Anexo: contexto (A1), casos de uso (A2), clases del dominio (A3), secuencia de elegir una opción (A4) y estados de una sesión (A5). Se usa el mismo vocabulario en requisitos, diagramas y código: *historia, escena, opción, condiciones, efectos, sesión, variables, final, bloqueo*. Por ejemplo, el método `Sesion.elegir` del diagrama A4 es el de `motor/sesion.py`, y los estados `EnCurso`, `Finalizada` y `Bloqueada` del diagrama A5 corresponden a las propiedades `terminada` y `bloqueada` de la clase `Sesion`. La guarda `[hay anterior]` del diagrama es `Sesion.puede_deshacer`. Los diagramas reflejan la versión final `v2.0`; los cambios respecto de `v1.0` se resumen en la sección 7. El mapa narrativo de la historia (A8) es un dato de entrada, no un modelo del software.

## 5. Arquitectura

El diagrama de componentes (Anexo A6) organiza el sistema en tres capas: **contenido** (`motor/modelo.py`, `motor/cargador.py`, `motor/validador.py`, `motor/errores.py`), **reglas** (`motor/sesion.py`) e **interfaz** (`interfaz/consola.py`, `jugar.py`). Las dependencias apuntan solo hacia abajo. La tercera decisión, ADR-03 (mecanismo de deshacer), está en la sección 7.

**ADR-01. Separar contenido, reglas e interfaz.** *Contexto:* la escritora debe cambiar el relato sin programar (N01) y la segunda historia debe funcionar sin tocar el motor. *Alternativas:* (a) un único script que lee el JSON e imprime con `if` por escena; (b) tres capas con modelo inmutable; (c) aplicación web MVC. *Elección:* (b), con interfaz de consola. *Consecuencias:* (+) el motor no contiene IDs ni variables de ninguna historia, lo que se demuestra cargando `el_faro.json` con otros IDs y variables; (+) `Sesion` se prueba sin consola, de modo que U1-U6 no dependen de entrada y salida; (+) cambiar a web solo reemplazaría `interfaz/`; (−) hay más archivos y la consola debe traducir números a IDs de opción.

**ADR-02. Validación completa antes de jugar, acumulando todos los problemas.** *Contexto:* la coordinadora no acepta cierres inesperados y la escritora necesita saber qué corregir (N06). *Alternativas:* (a) detenerse en el primer error; (b) validar de forma perezosa durante la partida; (c) usar la biblioteca `jsonschema`; (d) un validador propio que recorre todo el archivo y acumula los problemas. *Elección:* (d). *Consecuencias:* (+) la escritora ve todos los errores en una sola ejecución, cada uno con categoría, causa y ubicación (`escenas[E3].opciones[cortar].destino`); (+) el motor puede asumir datos válidos, por lo que no hay comprobaciones defensivas en `Sesion`; (+) no agrega dependencias; (−) el validador debe actualizarse si cambia el contrato. `jsonschema` se descartó porque no valida referencias cruzadas (destinos y variables declaradas) y sus mensajes son genéricos y en inglés.

**Separación demostrada con la segunda historia.** `historias/el_faro.json` usa IDs (`F1`…`F4`) y una variable (`tiene_llave`) distintos. Se juega con `python jugar.py historias/el_faro.json` sin modificar `motor/` ni `interfaz/` (RNF01, sección 8).

## 6. Implementación

**Tecnología.** Python 3.10+ con interfaz de consola. El programa usa solo la biblioteca estándar y las pruebas usan pytest. El código está versionado en Git; los tags `v1.0` y `v2.0` marcan las versiones anterior y posterior al cambio.

| Funcionalidad | Dónde se implementa |
|---|---|
| Carga desde archivo y construcción del modelo | `motor/cargador.py::cargar_historia`, `motor/modelo.py::historia_desde_datos` |
| Validación con las once categorías de error | `motor/validador.py::validar`, `motor/errores.py` |
| Opciones habilitadas, elección, estado, finales, bloqueo | `motor/sesion.py::Sesion` |
| Reinicio y deshacer | `Sesion.reiniciar`, `Sesion.deshacer` |
| Presentación, comandos y entradas inválidas | `interfaz/consola.py::Consola` |
| Argumentos, `--validar` y manejo de errores sin trazas | `jugar.py` |

**Decisión de diseño detallado: las operaciones devuelven `Resultado` en lugar de lanzar excepciones o imprimir.** `Sesion.elegir`, `deshacer` y `reiniciar` devuelven un objeto inmutable `Resultado(aceptada, mensaje)`. *Alternativas consideradas:* que el motor imprima directamente (acopla las reglas a la consola) o que lance excepciones ante una opción no habilitada (obliga a la interfaz a usar `try` para un caso normal de uso). *Consecuencias:* la sesión no depende de la interfaz, así que el mismo motor serviría para una interfaz web; las pruebas U3 y U6 verifican el rechazo con `assert resultado.aceptada is False` sin capturar salida; y una entrada inválida nunca interrumpe el flujo. Las excepciones se reservan para el contenido inválido (`ErrorContenido`), que sí impide abrir la sesión. Para reducir el acoplamiento también se tomaron otras dos medidas. Primera: `Consola` recibe por parámetro las funciones de entrada y salida, lo que permite que I1 y RNF03 la ejecuten con guiones. Segunda: el modelo es inmutable (`frozen=True` y `MappingProxyType`), de modo que `Sesion` es el único lugar donde cambia el estado y el deshacer pudo agregarse en un solo punto.

**Bibliotecas y licencias.** Biblioteca estándar de Python (PSF License): `json`, `dataclasses`, `types`, `argparse`, `pathlib`, `hashlib`, `random`, `subprocess`. pytest 8+ (MIT): pruebas, solo en desarrollo. Mermaid (MIT): solo para dibujar los diagramas del anexo; no forma parte del programa. Las versiones requeridas y los comandos de instalación y ejecución están en `README.md`.

## 7. Gestión del cambio: deshacer la última decisión (SC01)

**Versión de partida.** Al recibir la solicitud de la hora 6 la versión disponible era el tag **`v1.0`** (commit `d090e6c`). Estaba completa: 54 pruebas pasaban (`evidencias/registro_pruebas_v1.0.txt`) y no tenía retroceso, según el acuerdo provisional P1. La solicitud **SC01** reemplaza ese acuerdo y responde la pregunta P1: el retroceso es de **un solo nivel** y revierte todas las consecuencias.

**Nueva necesidad y requisitos.** *N07 (SC01):* el lector necesita deshacer su última decisión y que se reviertan todas sus consecuencias.

| RF | Requisito | Criterio de aceptación |
|---|---|---|
| RF15 | Deshacer restaura exactamente la escena y **todas** las variables que había antes de la última elección válida, también desde una escena final. | Desde `E2`, `investigar` lleva a `E3` con `conoce_origen = verdadero`; deshacer devuelve a `E2` con `conoce_origen = falso`. Desde `E4`, `E5` o `E6`, deshacer devuelve a `E3` con los valores previos. |
| RF16 | El retroceso es de un solo nivel: al comenzar no está disponible, y después de deshacer no se permite otro hasta realizar una nueva elección válida. | Un segundo deshacer seguido no cambia nada y se informa "No hay una decisión para deshacer"; tras `continuar`, deshacer vuelve a estar disponible. |
| RF17 | Reiniciar elimina el historial. Una opción inexistente o no habilitada no modifica la sesión ni sustituye la última elección válida guardada. | Tras reiniciar, deshacer es rechazado en `E1`. En `E3`, enviar `activar_sin_verificar` (no habilitada) y luego deshacer regresa a `E2`. |

**Análisis de impacto.**

| Artefacto | Impacto | Cambio realizado |
|---|---|---|
| Requisitos | Alto | Se agregan N07, RF15-RF17. P1 pasa de "acuerdo provisional" a "respondida por SC01". RF06 y RF10 no cambian su texto, pero su aceptación ahora también cubre el historial. |
| Casos de uso (A2) | Medio | Nuevo CU06 *Deshacer última decisión*. |
| Clases (A3) | Medio | `Sesion` agrega `_anterior: EstadoSesion \| None`, `puede_deshacer` y `deshacer()`. Nueva clase `EstadoSesion` (instantánea inmutable). |
| Secuencia (A4) | Bajo | `elegir` guarda la instantánea antes de aplicar los efectos. |
| Estados (A5) | Medio | Nueva transición "deshacer" desde `EnCurso`, `Finalizada` y `Bloqueada` cuando hay instantánea. |
| Componentes (A6) | Nulo | Las capas y dependencias no cambian; el contenido JSON tampoco (RNF01 se mantiene). |
| Código | Bajo | `motor/sesion.py` (+1 clase, +2 miembros) e `interfaz/consola.py` (comando `d`). Cargador, validador y modelo sin cambios. |
| Pruebas | Medio | Nueva U6 con variantes; se extienden U5 (reiniciar borra historial) e I1 (deshacer desde un final tras cargar el archivo). |
| Trazabilidad, README, backlog | Medio | Filas RF15-RF17 en la matriz (A7), comando `d` en el README y tarjetas B12-B13 en el tablero. |

**ADR-03. Mecanismo de deshacer: instantánea del estado previo.** *Contexto:* hay que restaurar **exactamente** la escena y todas las variables, con un solo nivel. *Alternativas:* (a) patrón Command, en el que cada efecto guarda su inverso; (b) pila de historial completa limitada a un elemento; (c) una **instantánea** inmutable (`EstadoSesion`) del estado anterior a la última elección válida. *Elección:* (c). *Consecuencias:* (+) la restauración es exacta por construcción, aunque un efecto asigne el valor que la variable ya tenía (caso en que un inverso calculado fallaría); (+) "un solo nivel" se cumple naturalmente: al deshacer, la instantánea pasa a `None`; (+) el cambio se concentra en `elegir`, que ya era el único punto que modifica el estado; (−) para rehacer o deshacer varios niveles habría que cambiar la instantánea por una pila (no se exige).

**Evidencia del comportamiento nuevo.** `tests/test_cambio_deshacer.py` (U6 y sus variantes), `tests/test_motor.py::test_u5_reiniciar_elimina_historial`, `tests/test_i1_integracion.py::test_i1_deshacer_desde_un_final_*` y el recorrido `evidencias/recorridos/deshacer.txt`. La versión posterior al cambio es el tag **`v2.0`**.

## 8. Verificación y resultados

**Pruebas automatizadas** (`python -m pytest -v`). La última ejecución da **70 pruebas aprobadas y 0 fallidas** (`evidencias/registro_pruebas.txt`). La versión `v1.0` tenía 54 pruebas aprobadas (`evidencias/registro_pruebas_v1.0.txt`). Cada caso declara entradas, resultado esperado y aserciones; las variantes se agrupan con `pytest.mark.parametrize`.

| Escenario | Archivo / prueba | Variantes |
|---|---|---|
| U1 Condiciones cumplidas y no cumplidas | `tests/test_motor.py::test_u1_*` | 2 de `E3` + 5 de evaluación |

| Escenario | Archivo / prueba | Variantes |
|---|---|---|
| U2 Elección válida: efectos y destino | `tests/test_motor.py::test_u2_*` | 6 caminos (incluye CA01) |
| U3 Opción inexistente o no habilitada | `tests/test_motor.py::test_u3_*` | 7 (incluye CA02 y elección desde un final) |
| U4 Errores de contenido | `tests/test_u4_validacion.py` | 11 archivos + 2 de lectura + 5 en memoria + acumulación |
| U5 Reiniciar (extendida por SC01) | `tests/test_motor.py::test_u5_*` | 5 de restauración + 3 de borrado de historial |
| U6 Deshacer (SC01) | `tests/test_cambio_deshacer.py` | Restaura y consume; inicio; entrada inválida ×4; nueva elección; un nivel; efecto idempotente |
| I1 Archivo real + motor (extendida por SC01) | `tests/test_i1_integracion.py` | 3 finales; deshacer desde los 3 finales; consola completa; consola con deshacer |

**Cobertura de validación.** Hay un caso por cada categoría exigida: archivo ilegible o estructura incorrecta, campos obligatorios ausentes, inicio inexistente, escenas u opciones duplicadas, destino inexistente, variable no declarada, valor no booleano y cantidad de opciones incompatible con el carácter final. Además se cubre el **bloqueo durante la partida** (`test_bloqueo_en_partida_*` y `evidencias/recorridos/bloqueo.txt`). El mensaje de CA03 es: `[DESTINO_INEXISTENTE] El destino 'E99' no corresponde a ninguna escena. (en escenas[E3].opciones[cortar].destino)`.

**Verificación de RNF** (`python scripts/verificar_rnf.py`; resultados reales en `evidencias/rnf_resultados.txt`).

| RNF | Meta | Resultado medido |
|---|---|---|
| RNF01 Modificabilidad | 0 líneas de código modificadas para historias nuevas | **0**. Hash SHA-256 de `motor/` e `interfaz/` idéntico antes y después. `el_faro.json` alcanza sus 2 finales (`F3`, `F4`) y la escena `E7` agregada por el script es jugable. El commit de la segunda historia solo modificó `historias/el_faro.json`. |
| RNF02 Claridad de errores | 100 % con categoría, ID y sin `Traceback` | **11/11 = 100 %**, con código de salida 1 en todos |
| RNF03 Confiabilidad | 0 excepciones y 0 alteraciones con 1000 entradas | Referencia: 1000 entradas, 871 rechazadas, **0 excepciones, 0 alteraciones**. `el_faro.json`: 1000 entradas, 855 rechazadas, **0 y 0** |

**Defecto encontrado por la verificación.** En la primera ejecución de RNF03, la entrada `'²'` cerraba la consola: `'²'.isdigit()` es verdadero en Python, pero `int('²')` lanza `ValueError`. Se corrigió en `interfaz/consola.py` (se exige `isascii()`) y se agregó una prueba de regresión (`test_rnf03_regresion_digitos_unicode_*`).

**Verificación del producto.** Los recorridos reales están en `evidencias/recorridos/`. Incluyen los tres finales (`final_E4.txt`, `final_E5.txt`, `final_E6.txt`), cómo una condición cambia las opciones (`condicion_cambia_opciones.txt`), entradas inválidas, reinicio, deshacer, la segunda historia, el bloqueo y el error `E99`. La suite también se ejecutó en un segundo equipo (`evidencias/ejecucion_segundo_equipo.txt`).

**Fallos pendientes y limitaciones declaradas.** No hay pruebas fallidas. Limitaciones: (1) la descripción técnica del JSON mal formado la genera Python en inglés, aunque la categoría, la línea y la columna están en español; (2) no se analizan todos los caminos del relato antes de jugar, por lo que un bloqueo se informa cuando ocurre (la guía no lo exige); (3) deshacer es de un nivel y sin rehacer, como pide SC01.

## 9. Aportes, fuentes y herramientas de apoyo

**Declaración de aportes** (verificable en `evidencias/historial_git.txt`, por autor de cada commit).

| Integrante | Aportes | Evidencia en el historial |
|---|---|---|
| Camilo Conde Corrales | Alcance, riesgos y proceso; requisitos y su corrección tras la revisión; modelos y arquitectura; implementación de las capas de contenido, reglas e interfaz; pruebas U1-U6 e I1; verificación de RNF; análisis e implementación del cambio SC01; README e informe | Commits de "Estructura inicial..." a "SC01: actualiza modelos..." y la generación de evidencias |
| Miguel Jacome y Yanez | Revisión cruzada de requisitos (detectó el defecto D1 de RF04); redacción final de la segunda historia; ejecución de pruebas y RNF en un segundo equipo; tablero Kanban y sus capturas; revisión del guion y grabación del video | Commits "Revisión cruzada...", "Ajusta redacción de El faro..." y "Ejecución en segundo equipo..." |

Ambos integrantes revisaron el informe final y prepararon la demostración.

**Fuentes.** Documentación oficial de Python 3 (`json`, `dataclasses`, `argparse`); documentación de pytest (parametrización y `tmp_path`); sintaxis de diagramas de Mermaid. No se reutilizaron fragmentos de código de tutoriales ni de otros proyectos.

**Herramientas de IA.** Se utilizó un asistente de IA conversacional (Claude, de Anthropic) para: (a) proponer alternativas de proceso, arquitectura, RNF y mecanismo de deshacer, que los integrantes evaluaron y aprobaron; (b) redactar borradores de la documentación y los diagramas; (c) generar una primera versión del código, las pruebas y los scripts de verificación. **Cómo se verificaron sus resultados:** cada artefacto se contrastó con la guía y con el anexo de la hora 6; se ejecutaron las 70 pruebas automatizadas y el script de RNF en dos equipos; se recorrieron a mano los tres finales, la segunda historia y el error `E99`; y la revisión cruzada (D1) y la verificación de RNF03 (dígitos Unicode) encontraron defectos que se corrigieron. El trabajo de los integrantes consistió en las decisiones, la revisión y la verificación; el material generado con apoyo de la herramienta se distingue en esta declaración.